Chapitre 5:
La Programmation Orientée Objet

Plan



1. Notions de base de la POO
2. Les notions de Classe et d’objet
3. La notion d’encapsulation
4. La notion d’héritage

Notions de base de la POO

Notions de base de la POO

- Nous avons vu plusieurs types de base en Python (int pour les entiers, float pour les flottants, str pour les chaînes de caractères, etc.).

- Les notions de **classe et d'objet** va nous permettre de généraliser la notion de « **type** » afin de créer de nouvelles structures de données un peu plus complexe et plus complètes.

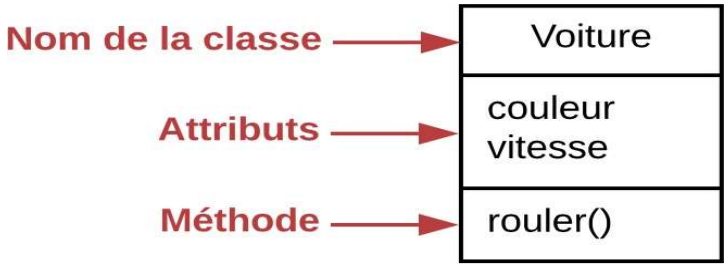
Les notions de Classe et d'Objet

La notion de Classe

- Une classe définit des **attributs** et des **méthodes**.

- Par exemple, imaginons une classe Voiture qui servira à créer des objets qui sont des voitures. Cette classe va pouvoir définir un attribut couleur, , un attribut vitesse, etc. Ces attributs correspondent à des propriétés qui peuvent exister pour une voiture.

- La classe Voiture pourra également définir une méthode rouler(). Une méthode correspond en quelque sorte à une action, ici l'action de rouler peut être réalisée pour une voiture.



La notion de Classe

- Si on imagine une classe **Avion**, elle pourra définir une méthode voler(). Elle pourra aussi définir une méthode rouler(). Par contre, la classe **Voiture** n'aura pas de méthode voler() car une voiture ne peut pas voler.

- De même, la classe Avion pourra avoir un attribut altitude mais ce ne sera pas le cas pour la classe Voiture.

➔ Une classe est représentative de la notion qu'elle met en place.

La notion d'objet

- On dit qu'un **objet est une instance de classe**.
- Soit la classe Voiture, nous pourrons avoir plusieurs voitures qui seront chacune des instances bien distinctes.
- Par exemple, la voiture de Rose, qui est de couleur rouge avec une vitesse de 300 km/h, est une instance de la classe Voiture, c'est un **objet**.
- De même, la voiture de Jeff, qui est de couleur grise avec une vitesse de 350 km/h, est un autre objet.
- Nous avons plusieurs objets pour une même classe, en particulier ici deux objets (autrement dit : deux instances de la même classe). Chacun des objets a des valeurs qui lui sont propres pour les attributs.

Définition d'une Classe

- Une classe appelée « Point » se définit de la manière suivante :

```
class Point:
    "définition d'un point géométrique"
```

- Par convention en Python, le nom identifiant une classe (qu'on appelle aussi son identifiant) débute par une majuscule. Ici **Point** débute par un **P** majuscule.

Création d'un objet de type Point

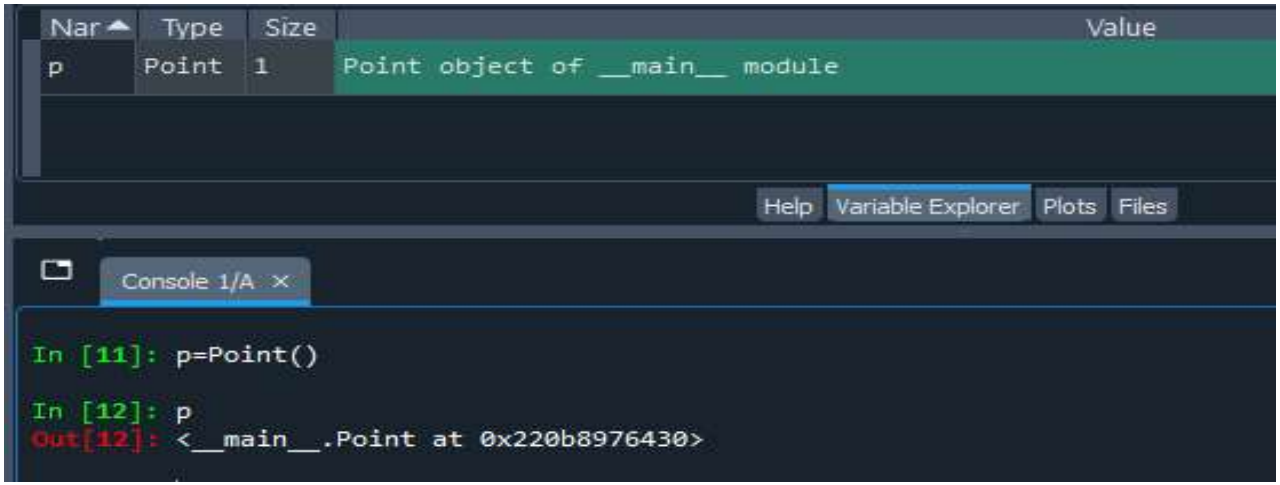
```
In [7]: Point()
Out[7]: <__main__.Point at 0x220b89b37f0>
```

- Ceci crée un objet de type Point. En POO, on dit que l'on crée une **instance** de la classe Point.
- Une phrase de la POO consiste à dire qu'**un objet est une instance de classe**.
- Il faut bien noter que pour créer une instance, **on utilise le nom de la classe suivi de parenthèses.**

Affectation à une variable de la référence à un objet

- Nous venons de définir une classe Point.

- Nous pouvons créer des objets de ce type, par instanciation. Créons par exemple un nouvel objet et mettons la référence à cet objet dans la variable p :



Affectation à une variable de la référence à un objet

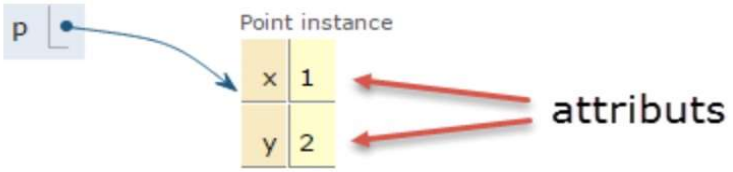
- Comme pour les fonctions, lors de l'appel à une classe dans une instruction pour créer un objet, il faut toujours indiquer des parenthèses (même si aucun argument n'est transmis). Nous verrons un peu plus loin que ces appels peuvent se faire avec des arguments (notion de constructeur).

- Remarquez bien cependant que la définition d'une classe ne nécessite pas de parenthèses (contrairement à ce qui de règle lors de la définition des fonctions).

Définition des attributs

```
class Point:
    "Definition d'un point geometrique"

p = Point()
p.x = 1
p.y = 2
print("p : x =", p.x, "y =", p.y)
```

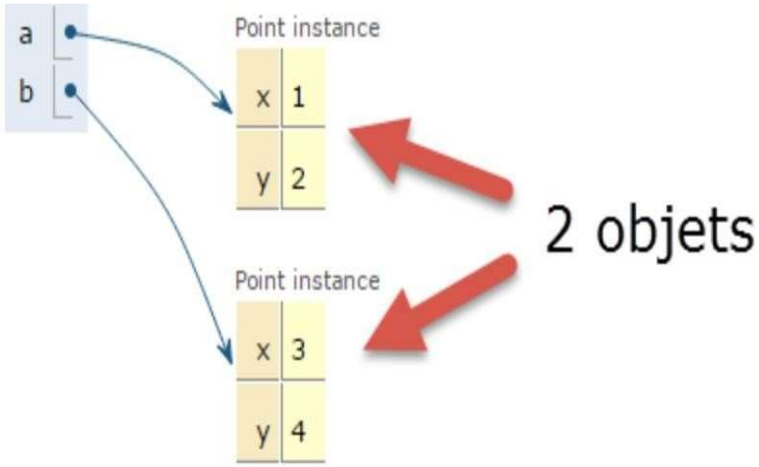


- La syntaxe pour accéder à un attribut est la suivante : on va utiliser la variable qui contient la référence à l'objet et on va mettre un point . puis le nom de l'attribut.

Définition des attributs

```
class Point:
    "Definition d'un point geometrique"

a = Point()
a.x = 1
a.y = 2
b = Point()
b.x = 3
b.y = 4
print("a : x =", a.x, "y =", a.y)
print("b : x =", b.x, "y =", b.y)
```



- On a 2 instances de la classe Point, c'est-à- dire 2 objets de type Point. Pour chacun d'eux, les attributs prennent des valeurs qui sont propres à l'instance.

Définition des attributs

```
class Point:
    "Definition d'un point geometrique"

a = Point()
a.x = 1
a.y = 2
b = a
print("a : x =", a.x, "y =", a.y)
print("b : x =", b.x, "y =", b.y)
a.x = 3
a.y = 4
print("a : x =", a.x, "y =", a.y)
print("b : x =", b.x, "y =", b.y)
```



- Les variables **a** et **b** font référence au même objet.
- L'affectation **b = a**, on met dans la variable **b** la référence contenue dans la variable **a**.

➔ toute modification des valeurs des attributs de l'objet dont la référence est contenue dans **a** entraîne une modification pour **b.**

Définition des méthodes

```
class Point:
    def deplace(self, dx, dy):
        self.x = self.x + dx
        self.y = self.y + dy
```

- Cette classe possède une méthode : **deplace()**
- Pour définir une méthode, il faut :
  - ✓ indiquer son nom (ici deplace())
  - ✓ indiquer les arguments entre des parenthèses. Le premier argument d'une méthode doit être **self**.

- **self** représente l'instance de la classe. En utilisant le mot-clé «**self**», nous pouvons accéder aux attributs et méthodes de la classe en python. Il lie les attributs avec les arguments donnés.

Définition des méthodes

- Pour accéder aux méthodes d'un objet, on indique :
  1. le nom de la variable qui fait référence à cet objet
  2. un point
  3. le nom de la méthode

```
a.deplace(3, 5)
```

- Lors de l'appel de la méthode, le paramètre **self** n'est pas utilisé et la valeur qu'il prend est la référence à l'objet. Il y a donc toujours un paramètre de moins que lors de la définition de la méthode.

```
class Point:
    "définition d'un point géométrique"
    def deplace(self,dx,dy):
        self.x = self.x + dx
        self.y = self.y + dy

p=Point()
p.x=1
p.y=2
print("p: x= ", p.x, " y= ", p.y)
p.deplace(2,2)
print("après déplacemnt: x= ", p.x, " y= ", p.y)
```

```
p: x=  1  y=  2
après déplacemnt: x=  3  y=  4
```

La notion de constructeur

- Si lors de la création d'un objet nous voulons qu'un certain nombre d'actions soit réalisées (par exemple une initialisation), nous pouvons utiliser un **constructeur**.

- Un constructeur n'est rien d'autre qu'une méthode, sans valeur de retour, qui porte un nom imposé par le langage Python : init ()

- Ce nom est constitué de init entouré avant et après par (**deux fois** le symbole **underscore** _).

- Cette méthode sera appelée lors de la création de l'objet. Le constructeur peut disposer d'un nombre quelconque de paramètres, éventuellement aucun.

La notion de constructeur

- Exemple sans paramètres :

```
class Point:
    def __init__(self):
        self.x=0
        self.y=0

a=Point()
print("a: x= ", a.x, " y= ", a.y)
a.x=1
a.y=2
print("a after first modif : x= ", a.x, " y= ", a.y)
```

```
a: x=  0  y=  0
a after first modif : x=  1  y=  2
```

La notion de constructeur

- Exemple avec paramètres :

```
class Point:
    def __init__(self, n1, n2):
        self.x = n1
        self.y = n2

a = Point(1, 2)
print("a : x =", a.x, "y =", a.y)
```

```
a : x = 1 y = 2
```

La notion de constructeur

Mot réservé **class**

Nom de la classe

- Exemple complet :

Méthode d'initialisation **__init__()**

self correspond à l'objet en cours

Définition des attributs

```
class Point:
    "définition d'un point géométrique"
    def __init__(self,x,y):
        self.x=x
        self.y=y

    def deplace(self,dx,dy):
        self.x = self.x + dx
        self.y = self.y + dy

a = Point(1, 2)
b = Point(3, 4)
print("a : x =", a.x, "y =", a.y)
print("b : x =", b.x, "y =", b.y)
a.deplace(3, 5)
b.deplace(-1, -2)
print("a après déplacement : x =", a.x, " y =", a.y)
print("b après déplacement : x =", b.x, " y =", b.y)
```

```
a : x = 1 y = 2
b : x = 3 y = 4
a après déplacement : x = 4  y = 7
b après déplacement : x = 2  y = 2
```

Application

1. Définissez une classe **Point3D** qui sera analogue à la classe Point mais pour des points dans l'espace à 3 dimensions.
2. Créez deux objets de type Point3D qui seront référencés par les variables a3D et b3D.
3. Initialiser ces points et afficher leurs coordonnées *x*, *y*, *z*.

Application

```python
class Point3D:
    def __init__(self,x,y,z):
        self.x=x
        self.y=y
        self.z=z

    def deplace(self,dx,dy,dz):
        self.x += dx
        self.y += dy
        self.z += dz

a3D=Point3D(1, 3, 2)
b3D=Point3D(4, 4, 4)
print("les coordonnées de a3D sont : ", a3D.x, a3D.y, a3D.z)
print("les coordonnées de b3D sont : ", b3D.x, b3D.y, b3D.z)
a3D.deplace(2, -2, -2)
print("les coordonnées de a3D après déplacement sont : ", a3D.x, a3D.y, a3D.z)
```

```
les coordonnées de a3D sont :  1 3 2
les coordonnées de b3D sont :  4 4 4
les coordonnées de a3D après déplacement sont :  3 1 0
```

La notion d’encapsulation

Définition d'attributs privés

- Le concept d'encapsulation est un concept très utile de la POO. Il permet en particulier d'éviter une modification par erreur des données d'un objet.

- En effet, il n'est alors pas possible d'agir directement sur les données d'un objet, il est nécessaire de passer par des méthodes qui jouent le rôle d'interface obligatoire

- **La protection des attributs d'une classe est réalisée grâce à l'utilisation d'attributs privées.**

- Pour avoir des attributs privés, leur nom doit débuter par __ (deux fois le symbole underscore _(touche 8)).

Définition d'attributs privés

- Exemple :

```
class Point:
    def __init__(self,x,y):
        self.__x=x
        self.__y=y
```

```
In [42]: p=Point(1,2)

In [43]: p.__x
Traceback (most recent call last):

  File "C:\Users\Sawssen\AppData\Local\Temp/ipykernel_11084/4124898122.py", line 1, in <modul
    p.__x

AttributeError: 'Point' object has no attribute '__x'
```

➔ Il n'est alors plus possible de faire appel aux attributs __x et __y depuis l'extérieur de la classe Point

Définition d'attributs privés

➔ Il faut donc disposer de méthodes qui vont permettre par exemple de modifier ou d'afficher les informations associées à ces variables.

```
class Point:
    def __init__(self, x, y):
        self.__x = x
        self.__y = y

    def deplace(self, dx, dy):
        self.__x = self.__x + dx
        self.__y = self.__y + dy

    def affiche(self):
        print("abscisse =", self.__x, "ordonnee =", self.__y)

a = Point(2, 4)
a.affiche()
a.deplace(1, 3)
a.affiche()
```

```
abscisse = 2 ordonnee = 4
abscisse = 3 ordonnee = 7
```

Accesseurs et mutateurs

- Parmi les différentes méthodes que comporte une classe, on a souvent tendance à distinguer :
  - les **constructeurs** ;
  - les **accesseurs** (en anglais *accessor*) qui fournissent des informations relatives à l'état d'un objet, c'est-à-dire aux valeurs de certains de ses attributs (généralement privés) sans les modifier ;
  - les **mutateurs** (en anglais *mutator*) qui modifient l'état d'un objet, donc les valeurs de certains de ses attributs.

Accesseurs et mutateurs

- On rencontre souvent l'utilisation de noms de la forme **get_XXXX()** pour les accesseurs et **set_XXXX()** pour les mutateurs, y compris dans des programmes dans lesquels les noms de variable sont francisés.

- Par exemple, pour la classe Point sur laquelle nous avons déjà travaillé on peut définir les méthodes suivantes :

Accesseurs et mutateurs

```python
class Point:
    def __init__(self, x, y):
        self.set_x(x)
        self.set_y(y)

    def get_x(self):
        return self.__x

    def set_x(self, x):
        self.__x = x

    def get_y(self):
        return self.__y

    def set_y(self, y):
        self.__y = y

a = Point(3, 7)
print("a : abscisse =", a.get_x())
print("a : ordonnee =", a.get_y())
a.set_x(6)
a.set_y(10)
print("a : abscisse =", a.get_x())
print("a : ordonnee =", a.get_y())
```

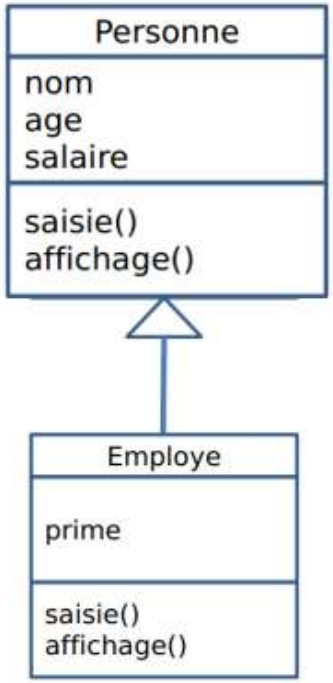
```
In [47]: runfile('C:/Users/S
python codes/cours POO')
a : abscisse = 3
a : ordonnee = 7
a : abscisse = 6
a : ordonnee = 10
```

La notion d’héritage

La notion d’héritage

- **Idée :** L’héritage permet de construire une hiérarchie de classes.
- Les classes héritières héritent des champs et méthodes de la classe ancêtre
- Ce mécanisme nécessite des efforts de modélisation et de conception. Mais au final, on améliore la lisibilité et la réutilisabilité du code
- L’héritage simple est le mécanisme permettant d’étendre une unique classe. Il consiste à créer une nouvelle classe (fille) qui bénéficiera des mêmes méthodes et attributs que sa classe mère. Il sera aisé d’en définir de nouveaux dans la classe fille, et cela n’altèrera pas le fonctionnement de la mère.

Héritage simple



La classe Employe est une Personne, avec le champ supplémentaire prime. Cela va nécessiter la reprogrammation des méthodes saisie() et affichage().
On peut éventuellement ajouter d'autres méthodes spécifiques à Employe.

Déclaration en Python

```
class Employe(Personne):
    ...
```

Héritage simple

```python
class Personne:
    #constructeur
    def __init__(self):
        #initialiser les attributs
        self.nom=""
        self.age=0
        self.salaire=0.0

    #méthode de saisie
    def saisie(self):
        self.nom=input('Donnez le nom : ')
        self.age=int(input('Donnez l âge : '))
        self.salaire=float(input('Donnez le salaire : '))

    #méthode d'affichage
    def affichage(self):
        print('le nom est : ' , self.nom)
        print('l âge est : ' , self.age)
        print('le salaie est : ' , self.salaire)
```

```
In [49]: p=Personne()

In [50]: p.saisie()

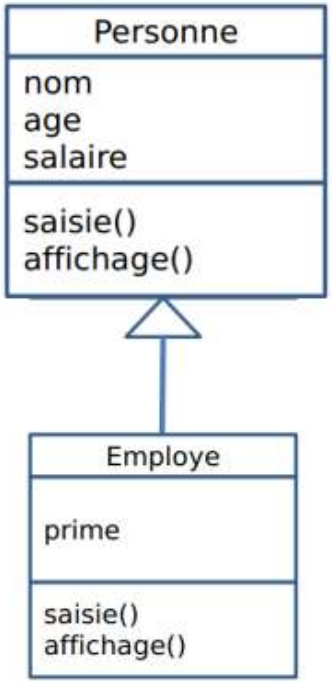
Donnez le nom : Yassine

Donnez l âge : 24

Donnez le salaire : 2000
```

```
In [52]: p.affichage()
le nom est :  Yassine
l âge est :  24
le salaie est :  2000.0
```

Héritage simple




```python
class Employe(Personne):
    #constructeur
    def __init__(self):
        Personne.__init__()
        self.prime=0.0
    #saisie
    def saisie(self):
        Personne.saisie(self)
        self.prime=float(input("Donnez prime : "))
    #affichage
    def affichage(self):
        Personne.affichage(self)
        print("Sa prime : ",self.prime)
```

```
In [69]: e=Employe()

In [70]: e.saisie()

Donnez le nom : yasmine

Donnez l âge : 25

Donnez le salaire : 2200

Donnez prime : 1000

In [71]: e.affichage()
le nom est :  yasmine
l âge est :  25
le salaie est :  2200.0
Sa prime :  1000.0
```

Héritage multiple

- Avec l'héritage simple, nous pouvions étendre le comportement d'une classe. L'héritage multiple va nous permettre de le faire pour plusieurs classes à la fois.

- Il nous suffit de préciser plusieurs classes entre parenthèses lors de la création de notre classe fille

```
class DerivedClassName(Base1, Base2, Base3):
    <statement-1>
    .
    .
    .
    <statement-N>
```

Application

Gestion du stock :

Un Article du stock est défini par 4 champs :
sa référence (numéro) . sa désignation (texte) . son prixHT . sa quantité (nombre d'articles disponibles)

1. Définissez la classe Article.
2. Ajouter une méthode qui permet de faire la saisie des attributs d'un objet
3. Ajouter une méthode qui permet de faire l'affichage des valeurs des attributs d'un objet
4. Développez les méthodes prixHT() et prixTTC() retournant respectivement (sous forme de réel) le prix Hors Taxes et le prix Toutes Taxes Comprises (avec un taux de TVA par défaut de 20%).
5. Développez une fonction qui permet d'ajouter un article au magasin (représenté par une liste).
6. Développez une fonction permettant de vérifier l'existence d'un article dans le magasin étant donnée sa désignation/référence
7. Ajouter une classe Vetement contenant les mêmes informations et services que la classe Article, avec en plus les attributs **taille** et **coloris**.